

P0260R8

C++ Concurrent Queues

Published Proposal, 2024-03-08

This version:

<http://wg21.link/P0260R8>

Authors:

Lawrence Crowl

Chris Mysen

[Detlef Vollmann](#)

Gor Nishanov

Audience:

SG1, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Concurrent queues are a fundamental structuring tool for concurrent programs. We propose a concurrent queue concept and a concrete implementation (in P1958). We propose a set of communication types that enable loosely bound program components to dynamically construct and safely share concurrent queues.

Table of Contents

1	Acknowledgments
2	Revision History
3	Introduction
3.1	Target Vehicle

3.1.1 Questions for a TS to Answer

4 Existing Practice

4.1 Concept of a Bounded Queue

4.2 Bounded Queues with C++ Interface

5 Conceptual Interface

5.1 Basic Operations

5.2 Asynchronous Operations

5.3 Non-Waiting Operations

5.4 Closed Queues

5.5 Element Type Requirements

5.6 Exception Handling

6 Concrete Queues

7 Proposed Wording

7.1 Concurrent Queues

7.1.1 General

7.1.2 Header `<experimental/conqueue>` synopsis

7.1.3 Concurrent Queues Concept

7.1.4 Class template `bounded_queue`

7.1.4.1 General

7.1.4.2 Modifiers

8 Old Revision History

9 Implementation

10 Historic Contents

10.1 Abandoned Interfaces

10.1.1 Re-opening a Queue

10.1.2 Non-Blocking Operations

10.1.3 Push Front Operations

10.1.4 Queue Names

10.1.5 Lock-Free Buffer Queue

10.1.6 Storage Iterators

- 10.1.7 Empty and Full Queues
- 10.1.8 Queue Ordering
- 10.1.9 Lock-Free Implementations
- 10.2 Abandoned Additional Conceptual Tools
 - 10.2.1 Fronts and Backs
 - 10.2.2 Streaming Iterators
 - 10.2.3 Binary Interfaces
 - 10.2.4 Managed Indirection

References

Informative References

§ 1. Acknowledgments

Thanks to David Goldblatt for his help with the wording!

§ 2. Revision History

This paper revises P0260R7 - 2023-06-15 as follows.

- Restructured document
- Fix typos
- Implement LEWG feedback to derive `conqueue_errc` from `system_error`
- Implement LEWG feedback to add range constructor and go back to `InputIterator`
- `size_t` `capacity()` added
- Added TBB `concurrent_bounded_queue` as existing practice
- Moved discussion about `pop()` interface to separate paper
- reintegrated P1958 `buffer_queue`
- Renamed `buffer_queue` to `bounded_queue`
- Added proposed wording (incomplete)
- Updated TS questions

Older revision history was moved to [after the proposed wording](#).

§ 3. Introduction

Queues provide a mechanism for communicating data between components of a system.

The existing `deque` in the standard library is an inherently sequential data structure. Its reference-returning element access operations cannot synchronize access to those elements with other queue operations. So, concurrent pushes and pops on queues require a different interface to the queue structure.

Moreover, concurrency adds a new dimension for performance and semantics. Different queue implementation must trade off uncontended operation cost, contended operation cost, and element order guarantees. Some of these trade-offs will necessarily result in semantics weaker than a serial queue.

Concurrent queues come in a several different flavours, e.g.

- bounded vs. unbounded
- blocking vs. overwriting
- single-ended vs. multi-ended
- strict FIFO ordering vs. priority based ordering

The syntactic concept proposed here should be valid for all of these flavours, while the concrete semantics might differ.

§ 3.1. Target Vehicle

This proposal targets a TS. It was originally sent to LEWG for inclusion into Concurrency TS v2. As Concurrency TS v2 will probably be published before this proposal is ready to be published, we propose to include concurrent queues into Concurrency TS v3 and publish this as soon as concurrent queues are ready. This leaves the door open for other proposal to share the same ship vehicle.

The scope for Concurrency TS v3 would be the same as that for v2:

"This document describes requirements for implementations of an interface that computer programs written in the C++ programming language may use to invoke algorithms with concurrent execution. The algorithms described by this document are realizable across a broad class of computer architectures."

Should the committee decide to restrict the scope of the TS to only contain concurrent queues, we propose a slightly different scope:

"This document describes requirements for implementations of an interface that computer programs written in the C++ programming language may use to communicate between different execution agents of algorithms with concurrent execution. The algorithms described by this document are realizable across a broad class of computer architectures."

§ 3.1.1. Questions for a TS to Answer

We expect that the TS will inform future work on a variety of questions, particularly those listed below, using real-world implementation experience that cannot be obtained without a TS.

- Is the proposed concept useful? Specifically, does it cover different implementations and does it work together with other concepts for concurrent queues, e.g. queues with only non-blocking functions or queues with an asynchronous interface?
- What other concrete implementations should be provided?
- Is the interface adequate for execution contexts from `std::execution` with weakly parallel forward progress?

§ 4. Existing Practice

§ 4.1. Concept of a Bounded Queue

The basic concept of a bounded queue with potentially blocking push and pop operations is very old and widely used. It's generally provided as an operating system level facility, like other concurrency primitives.

POSIX 2001 has `mq` message queues (with priorities and timeout).

Windows ?

FreeRTOS, Mbed, vxWorks

§ 4.2. Bounded Queues with C++ Interface

Literature

Boost

TBB has `concurrent_bounded_queue` (and an unbounded version `concurrent_queue` that has only non-blocking operations).

§ 5. Conceptual Interface

We provide basic queue operations, and then extend those operations to cover other important issues.

By analogy with how `future` defines their errors, we introduce `conque_errc` enum and `conqueue_error` as follows:

```
enum class conqueue_errc { success, empty, full, closed };

template <>
struct is_error_code_enum<conqueue_errc> : public true_type {};

const error_category& conqueue_category() noexcept;

error_code make_error_code(conqueue_errc e) noexcept;

error_condition make_error_condition(conqueue_errc e) noexcept;

class conqueue_error : public system_error;
```

These errors will be reported from concurrent queue operations as specified below.

§ 5.1. Basic Operations

The essential solution to the problem of concurrent queuing is to shift to value-based operations, rather than reference-based operations.

The basic operations are:

```
void queue::push(const T& x);
void queue::push(T&& x);
bool queue::push(const T& x, std::error_code& ec);
bool queue::push(T&& x, std::error_code& ec);
```

Pushes `x` onto the queue via copy or move construction. The first version throws `std::conqueue_error(conqueue_errc::closed)` if the queue is closed. The second version returns `true` on success, and `false` and sets `ec` to `error_code(conqueue_errc::closed)` if the queue is closed.

```
T queue::pop();  
std::optional<T> queue::pop(std::error_code& ec);
```

Pops a value from the queue via move construction into the return value. The first version throws `std::conqueue_error(conqueue_errc::closed)` if the queue is empty and closed; the second version, if the queue is empty and closed, returns `std::nullopt` and sets `ec` to `std::error_code(conqueue_errc::closed)`. If queue is empty and open, the operation blocks until an element is available.

In the original `buffer_queue` paper, the `pop` function had signature `T pop_value()`. Subsequently, it was changed to `void pop(T&)` due to concern about the problem of losing elements when an error occurs.

The exploration of different version of error reporting was moved to a separate paper [\[P2921R0\]](#).

§ 5.2. Asynchronous Operations

```
sender auto queue::async_push(T x);  
sender auto queue::async_pop();
```

These operations return a sender that will push or pop the element. Senders must support cancellation and if the receiver is currently waiting on a push or pop operation and no longer interested in performing the operation, it should be removed from any waiting queues, if any, and be completed with `std::execution::set_stopped`.

§ 5.3. Non-Waiting Operations

Waiting on a full or empty queue can take a while, which has an opportunity cost. Avoiding that wait enables algorithms to avoid queuing speculative work when a queue is full, to do other work rather

than wait for a push on a full queue, and to do other work rather than wait for a pop on an empty queue.

```
bool queue::try_push(const T& x, std::error_code& ec);  
bool queue::try_push(T&& x, std::error_code& ec);
```

If the queue is full or closed, returns `false` and sets the respective status in the `ec`. Otherwise, push the value onto the queue via copy or move construction and returns `true`.

REVISITED in Varna

The following version was introduced in response to LEWG-I concerns about losing the element if an rvalue cannot be stored in the queue.

```
queue_op_status queue::try_push(T&&, T&);
```

However, SG1 reaffirmed the APIs above with the following rationale:

It seems that it is possible in both versions:

```
T x = get_something();  
if (q.try_push(std::move(x))) ...
```

With two parameter version:

```
T x;  
if (q.try_push(get_something(), x)) ...
```

Ergonomically they are roughly identical. API is slightly simpler with one argument version, therefore, we reverted to original one argument version.

```
optional<T> queue::try_pop(std::error_code& ec);
```

If the queue is empty, returns `nullopt` and set `ec` to `conqueue_errc::empty`. Otherwise, pop the element from the queue via move construction into the optional. Return `true` and set `ec` {`.variable`} to `conqueue_errc::success`.

These operations will not wait when the queue is full or empty. They may block for mutual exclusion.

§ 5.4. Closed Queues

Threads using a queue for communication need some mechanism to signal when the queue is no longer needed. The usual approach is add an additional out-of-band signal. However, this approach suffers from the flaw that threads waiting on either full or empty queues need to be woken up when the queue is no longer needed. To do that, you need access to the condition variables used for full/empty blocking, which considerably increases the complexity and fragility of the interface. It also leads to performance implications with additional mutexes or atomics. Rather than require an out-of-band signal, we chose to directly support such a signal in the queue itself, which considerably simplifies coding.

To achieve this signal, a thread may `close` a queue. Once closed, no new elements may be pushed onto the queue. Push operations on a closed queue will either return `conqueue_errc::closed` (when they have `ec` parameter) or throw `conqueue_error(conqueue_errc::closed)` (when they do not). Elements already on the queue may be popped off. When a queue is empty and closed, pop operations will either set `ec` to `conqueue_errc::closed` (when they have a `ec` parameter) or throw `conqueue_error(conqueue_errc::closed)` otherwise.

The additional operations are as follows. They are essentially equivalent to the basic operations except that they return a status, avoiding an exception when queues are closed.

```
void queue::close() noexcept;
```

Close the queue.

```
bool queue::is_closed() const noexcept;
```

Return true iff the queue is closed.

§ 5.5. Element Type Requirements

The above operations require element types with copy/move constructors, and destructor. These operations may be trivial. The copy/move constructors operators may throw, but must leave the objects in a valid state for subsequent operations.

§ 5.6. Exception Handling

`push()` and `pop()` may throw an exceptions of type `conqueue_error` that's derived from `std::system_error` and will contain a `conqueue_errc`.

Concurrent queues cannot completely hide the effect of exceptions thrown by the element type, in part because changes cannot be transparently undone when other threads are observing the queue.

Queues may rethrow exceptions from storage allocation, mutexes, or condition variables.

If the element type operations required do not throw exceptions, then only the exceptions above are rethrown.

When an element copy/move may throw, some queue operations have additional behavior.

- Construction shall rethrow, destroying any elements allocated.
- A push operation shall rethrow and the state of the queue is unaffected.
- A pop operation shall rethrow and the element is popped from the queue. The value popped is effectively lost. (Doing otherwise would likely clog the queue with a bad element.)

§ 6. Concrete Queues

In addition to the concept, the standard needs at least one concrete queue. This paper proposes a fixed-size `bounded_queue`. It meets the concept of a concurrent queue. It provides for construction of an empty queue, and construction of a queue from an `initializer_list` or a pair of iterators. Constructors take a parameter specifying the maximum number of elements in the queue.

`bounded_queue` is only allowed to allocate in its constructor. Constructors that take an initializing sequence are allowed to omit the `size_t max_elem` argument that will be then assumed to be equal to the size of the initialization sequence.

§ 7. Proposed Wording

Add a new subclause to clause **[thread]** with the following contents.

§ 7.1. Concurrent Queues

§ 7.1.1. General

Concurrent queues provide a mechanism to transfer objects from one point in a program to another without producing data races.

§ 7.1.2. Header `<experimental/conqueue>` synopsis

```
namespace experimental {
    template <class T, class Q>
        concept concurrent_queue = see below;

    enum class conqueue_errc { success, empty, full, closed };

    const error_category& conqueue_category() noexcept;
    error_code make_error_code(conqueue_errc e) noexcept;
    error_condition make_error_condition(conqueue_errc e) noexcept;

    class conqueue_error : system_error { ... };

    template <typename T,
        class Allocator = std::allocator<T>>
        class bounded_queue;
}
```

§ 7.1.3. Concurrent Queues Concept

1. The `concurrent_queue` concept defines the requirements for a concurrent queue type.

```
template <class T, class Q>
    concept concurrent_queue =
        move_constructible<remove_cvref_t<T>> &&
        same_as<decay_t<T>, queue::value_type> &&
        requires (Q q, T &&t, std::error_code ec) {
            q.capacity() -> void; // still need to require const noexcept
            q.is_closed() -> bool; // still need to require const noexcept
            q.close() -> void; // still need to require noexcept
            q.push(std::forward<T>(t)) -> void;
```

```

q.push(std::forward<T>(t), ec) -> bool;
q.try_push(std::forward<T>(t), ec) -> bool;
q.async_push(const T&) -> std::execution::sender_of<void>;
q.async_push(T&&) -> std::execution::sender_of<void>;
q.pop() -> T;
q.pop(ec) -> std::optional<T>;
q.try_pop(ec) -> std::optional<T>;
q.async_pop() -> std::execution::sender_of<T>;
};

```

[Review note: this always requires `try_push`, even on an unbounded queue.]

2. In the following description, `Q` denotes a type conforming to the `concurrent_queue` concept, `q` denotes an object of type `Q` and `t` denotes an object convertible to the `Q::value_type`.
3. `push`, `try_push` and `async_push` are push operations.
4. `pop`, `try_pop` and `async_pop` are pop operations.
5. Calls to operations (except constructor and destructor) on the same queue from different threads of execution do not introduce data races.
6. Successful push and pop operations will call a constructor of `T`. For the description of concrete queues, *pre-ctor* is the call of the constructor and *post-ctor* is the return of the constructor. Likewise successful pop operations will call the destructor of `T` and we have *pre-dtor* and *post-dtor*.
7. A constructor or destructor called by a push or pop operation is required to return on the same thread of execution on which it was called.
8. In the following description, if an object is *deposited* into a queue by a push operation it can be *extracted* from the queue and returned by a pop operation for which the return *happens-after* the *post-ctor* of the push operation of the deposit. A pop operation on a queue can only extract objects that were deposited into the same queue and each object can only be extracted once.
9. Concrete queues shall specify whether a push may block for space available in the queue (bounded queue) or whether a push may allocate new space (unbounded queue). [Note: Even an unbounded queue is required to provide `async_push` and `try_push`. -- end note]
10. The expression `q.close()` has the following semantics:
 1. *Effects*: Closes the queue.
11. The expression `q.push(t)` has the following semantics:
 1. *Effects*: If the queue is not closed, `t` is deposited into `q`.

2. *Throws*: If the queue is closed throw `conqueue_error(conqueue_errc::closed)`. A concrete queue may throw additional exceptions.
12. The expression `q.try_push(t, ec)` has the following semantics:
 1. *Effects*: If the queue is not closed, and space is available in the queue, `t` is deposited into `q`. The operation will not block for space to be available in the queue. A concrete queue shall specify whether `try_push` may block for internal synchronization.
 2. *Returns*: `true` if `t` was deposited into `q`, otherwise `false`.
 13. The expression `q.async_push(t)` has the following semantics:
 1. *Returns*: A sender object `w` that behaves as follows:
 1. When `w` is connected with some receiver `r`, it returns an operation state `op` that behaves as follows:
 1. It waits until there's space in the queue or until the queue is closed.
 2. If there's space in the queue `t` will be deposited into the queue and `set_value(r)` will be called.
 3. If the queue is closed `set_stopped(r)` will be called.
 14. The expression `q.pop()` has the following semantics:
 1. *Effects*: Blocks the current thread until there's an object available in the queue or until the queue is closed. If the queue is closed, throw. Otherwise, if there is an object available in the queue, extract the object and return it.
 2. *Throws*: If the queue is closed throw `conqueue_error(conqueue_errc::closed)`. A concrete queue may throw additional exceptions.
 15. The expression `q.try_pop(ec)` has the following semantics:
 1. *Effects*: If there's an object available in the queue it will be extracted from the queue and returned. If the queue is closed an empty `optional` is returned. The operation will not block for an object to be available in the queue. A concrete queue shall specify whether `try_pop` may block for internal synchronization.
 2. *Returns*: `optional<T>(t)` if `t` was the available object in the queue, `optional<T>()` otherwise.
 16. The expression `q.async_pop()` has the following semantics:
 1. *Returns*: A sender object `w` that behaves as follows:

1. When `w` is connected with some receiver `r`, it returns an operation state `op` that behaves as follows:

1. It waits until there's an object available in the queue or until the queue is closed.
2. If there's an object `t` available in the queue it will be extracted and `set_value(r, t)` will be called.
3. If the queue is closed `set_stopped(r)` will be called.

§ 7.1.4. Class template `bounded_queue`

§ 7.1.4.1. General

1. A `bounded_queue` is a `concurrent_queue` and can hold a fixed size of objects which is given at construction time.

```
2. template <typename T,
    class Allocator = std::allocator<T>>
class bounded_queue
{
    bounded_queue() = delete;
    bounded_queue(const bounded_queue&) = delete;
    bounded_queue& operator=(const bounded_queue&) = delete;

public:
    typedef T value_type;

    // construct/destroy
    explicit bounded_queue(size_t max_elems, const Allocator& alloc = Allocator());
    explicit bounded_queue(std::initializer_list<T>, size_t max_elems = 0,
        const Allocator& alloc = Allocator());
    template <typename InputIterator>
        bounded_queue(InputIterator begin, InputIterator end, size_t max_elems,
            const Allocator& alloc = Allocator());
    template <container-compatible-range<T> R>
        bounded_queue(from_range_t, R&& rg, size_t max_elems = 0,
            const Allocator& alloc = Allocator());

    ~bounded_queue() noexcept;

    // observers
    size_t capacity() const noexcept;
```

```

bool is_closed() const noexcept;
static constexpr bool is_always_lock_free = implementation-defined;

// modifiers
void close() noexcept;

void push(const T& x);
void push(T&& x);
bool push(const T& x, std::error_code& ec);
bool push(T&& x, std::error_code& ec);
bool try_push(const T& x, std::error_code& ec);
bool try_push(T&& x, std::error_code& ec);
sender auto async_push(const T&);
sender auto async_push(T&&);

T pop();
optional<T> pop(std::error_code& ec);
optional<T> try_pop(std::error_code& ec);
sender auto async_pop();
};

```

```
static constexpr bool is_always_lock_free;
```

3. The static data member `is_always_lock_free` is `true` if the `try_*` operations are always lock-free, and `false` otherwise.

§ 7.1.4.2. Modifiers

```

void push(const T& x);
void push(T&& x);
bool push(const T& x, std::error_code& ec);
bool push(T&& x, std::error_code& ec);

```

4. *Effects*: Blocks the current thread until there's space in the queue or until the queue is closed.
5. Let *Push1* and *Push2* be push operations and *Pop1* and *Pop2* be pop operations, where *Pop1* returns the value of the parameter given to *Push1* and *Pop2* returns the value of the parameter given to *Push2*, then there is a total order consistent with `memory_order::seq_cst` of *pre-ctor*, *post-*

ctor, *pre-dtor* and *post-dtor* of *Push1*, *Push2*, *Pop1* and *Pop2*, and moreover if *post-ctor* of *Push1* is before *post-ctor* of *Push2* in the order, then *pre-ctor* of *Pop1* happens before *pre-ctor* of *Pop2*.

6. [Note: This guarantees FIFO behaviour, but for two concurrent pushes the constructors can not determine the order in which the values are enqueued, and the constructors can run concurrently as well. -- end note]
7. [Note: This does not guarantee that constructors or destructors may ever run concurrently. An implementation may decide that two pushes (or to pops) never run concurrently. -- end note]
8. [Note: A constructor can deadlock if it tries a push or pop on the same queue. -- end note]

```
T pop();  
optional<T> pop(std::error_code& ec);  
optional<T> try_pop(std::error_code& ec);  
sender auto async_pop();
```

9. *Remarks:* The constructor of the returned object and the destructor of the internal object run on the same thread of execution.

```
bool try_push(const T& x, std::error_code& ec);  
bool try_push(T&& x, std::error_code& ec);  
optional<T> try_pop(std::error_code& ec);
```

10. *Remarks:* These operations may block for internal synchronization.

§ 8. Old Revision History

This paper revises P0260R7 - 2023-06-15 as follows.

- Fix typos.
- Implement LEWG feedback to derive `conqueue_errc` from `system_error`
- Implement LEWG feedback to add range constructor and go back to `InputIterator`
- `size_t capacity()` added
- Added TBB `concurrent_bounded_queue` as existing practice
- Moved discussion about `pop()` interface to separate paper

P0260R6 revises P0260R5 - 2023-01-15 as follows.

- Fixing typos.
- Added a scope for the target TS.
- Added questions to be answered by a TS.
- Added asynchronous interface

P0260R5 revises P0260R4 - 2020-01-12 as follows.

- Added more introductory material.
- Added response to feedback by LEWGI at Prague meeting 2020.
- Added section on existing practice.
- Replaced `value_pop` with `pop`.
- Replaced `is_lock_free` with `is_always_lockfree`.
- Removed `is_empty` and `is_full`.
- Added move-into parameter to `try_push(Element&&)`
- Added note that exception thrown by the queue operations themselves are derived from `std::exception`.
- Added a note that the wording is partly invalid.
- Moved more contents into the "Abandoned" part to avoid confusion.

P0260R4 revised P0260R3 - 2019-01-20 as follows.

- Remove the binding of `queue_op_status::success` to a value of zero.
- Correct stale use of the `Queue` template parameter in `shared_queue_front` to `Value`.
- Change the return type of `share_queue_ends` from a `pair` to a custom struct.
- Move the concrete queue proposal to a separate paper, [\[P1958R0\]](#).

P0260R3 revised P0260R2 - 2017-10-15 as follows.

- Convert `queue_wrapper` to a `function`-like interface. This conversion removes the `queue_base` class. Thanks to Zach Lane for the approach.
- Removed the requirement that element types have a default constructor. This removal implies that statically sized buffers cannot use an array implementation and must grow a vector implementation to the maximum size.
- Added a discussion of checking for output iterator end in the wording.
- Fill in synopsis section.

- Remove stale discussion of `queue_owner`.
- Move all abandoned interface discussion to a new section.
- Update paper header to current practice.

P0260R2 revised P0260R1 - 2017-02-05 as follows.

- Emphasize that non-blocking operations were removed from the proposed changes.
- Correct syntax typos for `noexcept` and template alias.
- Remove `static` from `is_lock_free` for `generic_queue_back` and `generic_queue_front`.

P0260R1 revised P0260R0 - 2016-02-14 as follows.

- Remove pure virtuals from `queue_wrapper`.
- Correct `queue::pop` to `value_pop`.
- Remove nonblocking operations.
- Remove non-locking buffer queue concrete class.
- Tighten up push/pop wording on closed queues.
- Tighten up push/pop wording on synchronization.
- Add note about possible non-FIFO behavior.
- Define `buffer_queue` to be FIFO.
- Make wording consistent across attributes.
- Add a restriction on element special methods using the queue.
- Make `is_lock_free()` for only non-waiting functions.
- Make `is_lock_free()` static for non-indirect classes.
- Make `is_lock_free()` `noexcept`.
- Make `has_queue()` `noexcept`.
- Make destructors `noexcept`.
- Replace "throws nothing" with `noexcept`.
- Make the remarks about the usefulness of `is_empty()` and `is_full` into notes.
- Make the non-static member functions `is_...` and `has_...` functions `const`.

P0260R0 revised N3533 - 2013-03-12 as follows.

- Update links to source code.
- Add wording.
- Leave the name facility out of the wording.
- Leave the push-front facility out of the wording.
- Leave the reopen facility out of the wording.
- Leave the storage iterator facility out of the wording.

N3532 revised N3434 = 12-0043 - 2012-01-14 as follows.

- Add more exposition.
- Provide separate non-blocking operations.
- Add a section on the lock-free queues.
- Argue against push-back operations.
- Add a cautionary note on the usefulness of `is_closed()`.
- Expand the cautionary note on the usefulness of `is_empty()`. Add `is_full()`.
- Add a subsection on element type requirements.
- Add a subsection on exception handling.
- Clarify ordering constraints on the interface.
- Add a subsection on a lock-free concrete queue.
- Add a section on content iterators, distinct from the existing streaming iterators section.
- Swap front and back names, as requested.
- General expository cleanup.
- Add an "Revision History" section.

N3434 revised N3353 = 12-0043 - 2012-01-14 as follows.

- Change the inheritance-based interface to a pure conceptual interface.
- Put `try_...` operations into a separate subsection.
- Add a subsection on non-blocking operations.
- Add a subsection on push-back operations.
- Add a subsection on queue ordering.

- Merge the "Binary Interface" and "Managed Indirection" sections into a new "Conceptual Tools" section. Expand on the topics and their rationale.
- Add a subsection to "Conceptual Tools" that provides for type erasure.
- Remove the "Synopsis" section.
- Add an "Implementation" section.

§ 9. Implementation

An implementation is available at github.com/GorNishanov/conqueue.

A free, open-source implementation of an earlier version of these interfaces is available at the Google Concurrency Library project at github.com/aldasdairmackintosh/google-concurrency-library. The original `buffer_queue` is in `.../blob/master/include/buffer_queue.h`. The concrete `lock_free_buffer_queue` is in `.../blob/master/include/lock_free_buffer_queue.h`. The corresponding implementation of the conceptual tools is in `.../blob/master/include/queue_base.h`.

§ 10. Historic Contents

The Contents in this section is for historic reference only.

§ 10.1. Abandoned Interfaces

§ 10.1.1. Re-opening a Queue

There are use cases for opening a queue that is closed. While we are not aware of an implementation in which the ability to reopen a queue would be a hardship, we also imagine that such an implementation could exist. Open should generally only be called if the queue is closed and empty, providing a clean synchronization point, though it is possible to call open on a non-empty queue. An open operation following a close operation is guaranteed to be visible after the close operation and the queue is guaranteed to be open upon completion of the open call. (But of course, another close call could occur immediately thereafter.)

```
void queue::open();
```

Open the queue.

Note that when `is_closed()` returns false, there is no assurance that any subsequent operation finds the queue closed because some other thread may close it concurrently.

If an open operation is not available, there is an assurance that once closed, a queue stays closed. So, unless the programmer takes care to ensure that all other threads will not close the queue, only a return value of true has any meaning.

Given these concerns with reopening queues, we do not propose wording to reopen a queue.

§ 10.1.2. Non-Blocking Operations

For cases when blocking for mutual exclusion is undesirable, one can consider non-blocking operations. The interface is the same as the try operations but is allowed to also return

`queue_op_status::busy` in case the operation is unable to complete without blocking.

```
queue_op_status queue::nonblocking_push(const Element&);\ queue_op_status  
queue::nonblocking_push(Element&&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is full, return `queue_op_status::full`. Otherwise, push the `Element` onto the queue. Return `queue_op_status::success`.

```
queue_op_status queue::nonblocking_pop(Element&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is empty, return `queue_op_status::empty`. Otherwise, pop the `Element` from the queue. The element will be moved out of the queue in preference to being copied. Return `queue_op_status::success`.

These operations will neither wait nor block. However, they may do nothing.

The non-blocking operations highlight a terminology problem. In terms of synchronization effects, `nonwaiting_push` on queues is equivalent to `try_lock` on mutexes. And so one could conclude that the existing `try_push` should be renamed `nonwaiting_push` and `nonblocking_push` should be renamed `try_push`. However, at least Thread Building Blocks uses the existing terminology. Perhaps better is to not use `try_push` and instead use `nonwaiting_push` and `nonblocking_push`.

****In November 2016, the Concurrency Study Group chose to defer non-blocking operations. Hence, the proposed wording does not include these functions. In addition, as these functions were the only ones that returned `busy`, that enumeration is also not included.****

§ 10.1.3. Push Front Operations

Occasionally, one may wish to return a popped item to the queue. We can provide for this with `push_front` operations.

```
void queue::push_front(const Element&);\ void queue::push_front(Element&&);
```

Push the `Element` onto the back of the queue, i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.

```
queue_op_status queue::try_push_front(const Element&);\ queue_op_status  
queue::try_push_front(Element&&);
```

If the queue was full, return `queue_op_status::full`. Otherwise, push the `Element` onto the front of the queue, i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.

```
queue_op_status queue::nonblocking_push_front(const Element&);\  
queue_op_status queue::nonblocking_push_front(Element&&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is full, return `queue_op_status::full`. Otherwise, push the `Element` onto the front queue. i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.

This feature was requested at the Spring 2012 meeting. However, we do not think the feature works.

- The name `push_front` is inconsistent with existing `"push back"` nomenclature.
- The effects of `push_front` are only distinguishable from a regular push when there is a strong ordering of elements. Highly concurrent queues will likely have no strong ordering.
- The `push_front` call may fail due to full queues, closed queues, etc. In which case the operation will suffer contention, and may succeed only after interposing push and pop operations. The consequence is that the original push order is not preserved in the final pop order. So, `push_front` cannot be directly used as an `'undo'`.
- The operation implies an ability to reverse internal changes at the front of the queue. This ability implies a loss efficiency in some implementations.

In short, we do not think that in a concurrent environment `push_front` provides sufficient semantic value to justify its cost. Consequently, the proposed wording does not provide this feature.

§ 10.1.4. Queue Names

It is sometimes desirable for queues to be able to identify themselves. This feature is particularly helpful for run-time diagnostics, particularly when 'ends' become dynamically passed around between threads. See [Managed Indirection](#).

```
const char* queue::name();
```

Return the name string provided as a parameter to queue construction.

There is some debate on this facility, but we see no way to effectively replicate the facility. However, in recognition of that debate, the wording does not provide the name facility.

§ 10.1.5. Lock-Free Buffer Queue

We provide a concrete concurrent queue in the form of a fixed-size `lock_free_buffer_queue`. It meets the `NonWaitingConcurrentQueue` concept. The queue is still under development, so details may change.

In November 2016, the Concurrency Study Group chose to defer lock-free queues. Hence, the proposed wording does not include a concrete lock-free queue.

§ 10.1.6. Storage Iterators

In addition to iterators that stream data into and out of a queue, we could provide an iterator over the storage contents of a queue. Such an iterator, even when implementable, would mostly likely be valid only when the queue is otherwise quiescent. We believe such an iterator would be most useful for debugging, which may well require knowledge of the concrete class. Therefore, we do not propose wording for this feature.

§ 10.1.7. Empty and Full Queues

It is sometimes desirable to know if a queue is empty.

```
bool queue::is_empty() const noexcept;
```

Return true iff the queue is empty.

This operation is useful only during intervals when the queue is known to not be subject to pushes and pops from other threads. Its primary use case is assertions on the state of the queue at the end of its

lifetime, or when the system is in quiescent state (where there no outstanding pushes).

We can imagine occasional use for knowing when a queue is full, for instance in system performance polling. The motivation is significantly weaker though.

```
bool queue::is_full() const noexcept;
```

Return true iff the queue is full.

Not all queues will have a full state, and these would always return false.

§ 10.1.8. Queue Ordering

The conceptual queue interface makes minimal guarantees.

- The queue is not empty if there is an element that has been pushed but not popped.
- A push operation *synchronizes with* the pop operation that obtains that element.
- A close operation *synchronizes with* an operation that observes that the queue is closed.
- There is a sequentially consistent order of operations.

In particular, the conceptual interface does not guarantee that the sequentially consistent order of element pushes matches the sequentially consistent order of pops. Concrete queues could specify more specific ordering guarantees.

§ 10.1.9. Lock-Free Implementations

Lock-free queues will have some trouble waiting for the queue to be non-empty or non-full. Therefore, we propose two closely-related concepts. A full concurrent queue concept as described above, and a non-waiting concurrent queue concept that has all the operations except `push`, `wait_push`, `value_pop` and `wait_pop`. That is, it has only non-waiting operations (presumably emulated with busy wait) and non-blocking operations, but no waiting operations. We propose naming these `WaitingConcurrentQueue` and `NonWaitingConcurrentQueue`, respectively.

NOTE: Adopting this conceptual split requires splitting some of the facilities defined later.

For generic code it's sometimes important to know if a concurrent queue has a lock free implementation.

```
constexpr static bool queue::is_always_lock_free() noexcept;
```

Return true iff the has a lock-free implementation of the non-waiting operations.

§ 10.2. Abandoned Additional Conceptual Tools

There are a number of tools that support use of the conceptual interface. These tools are not part of the queue interface, but provide restricted views or adapters on top of the queue useful in implementing concurrent algorithms.

§ 10.2.1. Fronts and Backs

Restricting an interface to one side of a queue is a valuable code structuring tool. This restriction is accomplished with the classes `generic_queue_front` and `generic_queue_back` parameterized on the concrete queue implementation. These act as pointers with access to only the front or the back of a queue. The front of the queue is where elements are popped. The back of the queue is where elements are pushed.

```
void send( int number, generic_queue_back<buffer_queue<int>> arv );
```

These fronts and backs are also able to provide `begin` and `end` operations that unambiguously stream data into or out of a queue.

§ 10.2.2. Streaming Iterators

In order to enable the use of existing algorithms streaming through concurrent queues, they need to support iterators. Output iterators will push to a queue and input iterators will pop from a queue. Stronger forms of iterators are in general not possible with concurrent queues.

Iterators implicitly require waiting for the advance, so iterators are only supportable with the `WaitingConcurrentQueue` concept.

```
void iterate(  
    generic_queue_back<buffer_queue<int>>::iterator bitr,  
    generic_queue_back<buffer_queue<int>>::iterator bend,  
    generic_queue_front<buffer_queue<int>>::iterator fitr,  
    generic_queue_front<buffer_queue<int>>::iterator fend,  
    int (*compute)( int ) )  
{
```

```

while ( fitr != fend && bitr != bend )
    *bitr++ = compute(*fitr++);
}

```

Note that contrary to existing iterator algorithms, we check both iterators for reaching their end, as either may be closed at any time.

Note that with suitable renaming, the existing standard front insert and back insert iterators could work as is. However, there is nothing like a pop iterator adapter.

§ 10.2.3. Binary Interfaces

The standard library is template based, but it is often desirable to have a binary interface that shields client from the concrete implementations. For example, `std::function` is a binary interface to callable object (of a given signature). We achieve this capability in queues with type erasure.

We provide a `queue_base` class template parameterized by the value type. Its operations are virtual. This class provides the essential independence from the queue representation.

We also provide `queue_front` and `queue_back` class templates parameterized by the value types. These are essentially `generic_queue_front<queue_base<Value>>` and `generic_queue_back<queue_base<Value>>`, respectively.

To obtain a pointer to `queue_base` from an non-virtual concurrent queue, construct an instance the `queue_wrapper` class template, which is parameterized on the queue and derived from `queue_base`. Upcasting a pointer to the `queue_wrapper` instance to a `queue_base` instance thus erases the concrete queue type.

```

extern void seq_fill( int count, queue_back<int> b );

buffer_queue<int> body( 10 /*elements*/, /*named*/ "body" );
queue_wrapper<buffer_queue<int>> wrap( body );
seq_fill( 10, wrap.back() );

```

§ 10.2.4. Managed Indirection

Long running servers may have the need to reconfigure the relationship between queues and threads. The ability to pass 'ends' of queues between threads with automatic memory management eases

programming.

To this end, we provide `shared_queue_front` and `shared_queue_back` template classes. These act as reference-counted versions of the `queue_front` and `queue_back` template classes.

The `share_queue_ends(Args ... args)` template function will provide a pair of `shared_queue_front` and `shared_queue_back` to a dynamically allocated `queue_object` instance containing an instance of the specified implementation queue. When the last of these fronts and backs are deleted, the queue itself will be deleted. Also, when the last of the fronts or the last of the backs is deleted, the queue will be closed.

```
auto x = share_queue_ends<buffer_queue<int>>>( 10, "shared" );
shared_queue_back<int> b(x.back());
shared_queue_front<int> f(x.front());
f.push(3);
assert(3 == b.value_pop());
```

§ References

§ Informative References

[P1958R0]

Lawrence Crowl. *C++ Concurrent Buffer Queue*. 13 January 2020. URL: <https://wg21.link/p1958r0>

[P2921R0]

Gor Nishanov, Detlef Vollmann. *Exploring std::expected based API alternatives for buffer_queue*. 5 July 2023. URL: <https://wg21.link/p2921r0>